

AlgoVision 2.0 — Video Demonstration Script

Word-for-Word 5-10 Min Presentation & Walkthrough Guide

Date: October 2026
Standard: NASSCOM
FutureSkills
Candidate: Saikat Koner

LaunchED Capstone Project Demonstration Video Script & Guide

Project: AlgoVision 2.0 — Unified DSA Analytics & Optimization Platform

Submission Requirement: 5 to 10-Minute Video Demonstration

Format: MP4 Format (Uploaded to Google Drive with "Anyone with link can view" permission)

LinkedIn Sharing: Post video on LinkedIn tagging LaunchED and your HOD.

Section 1: Introduction (Mandatory in English)

Record this initial section with your webcam enabled, clear lighting, and confident delivery.

Word-for-Word Introduction Script:

"Hello everyone! My name is [Your Full Name], and I am currently pursuing my [B.Tech / B.E. / MCA / B.Sc] in [Computer Science and Engineering / Information Technology] from [Your College / University Name]. I am currently in my [3rd Year / 4th Year / Final Year] of study.

I have completed my internship journey with LaunchED in the [Data Structures & Algorithms / Full Stack Development / Software Engineering] domain. It has been an incredible learning experience where I gained deep exposure to corporate-grade coding standards, algorithmic problem-solving, and scalable system design.

In this video, I will be demonstrating my Capstone Major Project titled 'AlgoVision 2.0 — A Unified Analytics & Optimization Platform' that integrates core Data Structures and Algorithms to solve real-world challenges across logistics, e-commerce, and fintech sectors."

Section 2: Technical Project Walkthrough (English or Hindi)

You can present this section in English or Hindi based on your comfort level. Share your screen showing the web dashboard and VS Code.

Part 1: Problem Statement & Motivation (Duration: ~1.0 Min)

"Let us first understand the problem statement.

In modern enterprise software, naive algorithmic approaches lead to catastrophic latency spikes and memory bloat when scaling to millions of users.

For example:

1. In **Supply Chain Logistics**, unoptimized routing and freight packing waste millions of dollars in transit.
2. In **E-Commerce**, slow linear catalog searches and cache misses ruin user conversion.
3. In **FinTech**, failing to detect circular transaction loops in real-time allows sophisticated money laundering syndicates to go unnoticed.

To solve these three critical industry problems, I built **AlgoVision 2.0** — a unified platform integrating 5 core Data Structure & Algorithm modules aligned with the NASSCOM FutureSkills framework."

Part 2: Tools, Technologies & Architecture (Duration: ~1.5 Min)

"Let us look at the tools and architecture used in AlgoVision 2.0:

- **Core Language:** Python 3.10+ for the backend data structures, algorithms, and service engines.
- **Frontend Interface:** Pure JavaScript (ES6+), HTML5 Canvas, SVG, and handcrafted CSS3 design system with full Dark and Light theme support.
- **Testing Framework:** Python `unittest` suite with 24 comprehensive unit and integration tests passing at 100%.
- **Architecture:** Modular 5-tier architecture:
 1. **Data Ingestion:** Circular FIFO Queues and Hash Tables with Separate Chaining.
 2. **Cataloging:** Contiguous Dynamic Arrays with Yaroslavskiy Dual-Pivot QuickSort.
 3. **Search Engine:** Self-Balancing AVL Binary Search Trees with LL/RR/LR/RL rotations and Streaming Top-K Heaps.
 4. **Optimizer:** Greedy Algorithms (Dijkstra, Fractional Knapsack, Huffman) and Dynamic Programming (0/1 Knapsack, LCS, Floyd-Warshall Arbitrage).
 5. **Topology & Graphs:** Adjacency List representations with BFS, DFS, Tarjan's Strongly Connected Components, and Kruskal's Minimum Spanning Tree."

Part 3: Live Interactive Demonstration (Duration: ~4.0 Min)

During this section, show the live browser dashboard (`web/index.html`) and click through each tab.

1. Module 1 Demo — Data Ingestion:

"Here on the screen is Module 1: Data Ingestion. You can see our Circular FIFO Queue with capacity 8. When I click '+ Ingest Event', an event enters at the Tail pointer in $O(1)$ time. When I click '- Dispatch Event', the Head pointer advances in $O(1)$ without any expensive array shifting. Below it, our Transaction Deduplication Hash Table demonstrates separate chaining collision resolution and sub-microsecond idempotency verification."

2. Module 2 Demo — Cataloging & Sorting:

"Switching to Module 2: Cataloging. Here we visualize Yaroslavskiy's Dual-Pivot QuickSort on dynamic product arrays. When I click 'Play', notice how the algorithm selects two pivots — Pivot 1 in amber and Pivot 2 in purple — and partitions the array into three segments simultaneously. This achieves ~35% fewer comparisons than standard MergeSort and reduces CPU branch mispredictions."

3. Module 3 Demo — Search Engine (AVL Tree):

"In Module 3, we have our Self-Balancing AVL Binary Search Tree. Watch what happens when I insert a new price key: if the balance factor exceeds $+1$ or -1 , the engine automatically triggers Left or Right rotations to rebalance the tree in real-time. When I run a price range query between ₹20,000 and ₹60,000, it highlights all matching SKU nodes in $O(\log N + K)$ time, completely outperforming linear scans."

4. Module 4 Demo — Optimizer (0/1 DP Knapsack):

"In Module 4: Optimizer, we solve the Freight Truck Cargo Packing problem. Here is the full 2D Dynamic Programming matrix filling cell by cell according to the recurrence relation: $DP[i][w] = \max(DP[i-1][w], \text{val}[i] + DP[i-1][w - \text{wt}[i]])$. The green highlighted cells show the exact discrete items selected through backtrack path reconstruction."

5. Module 5 Demo — Graph Topology & Fraud Rings:

"In Module 5: Network Topology, we have three industry graph modes:

1. Logistics Dijkstra Routing: Computes the shortest delivery route from Mumbai to Kolkata across our supply chain network.
2. Kruskal's MST: Uses Disjoint Set Union (DSU) to connect regional warehouses with the minimum possible fiber-optic cabling distance.
3. FinTech Fraud Ring Detection: Applies Tarjan's Strongly Connected Components to identify circular money-laundering rings, highlighting suspicious accounts in glowing red."

6. Live In-Browser Benchmarks:

"Finally, on the Live Benchmarks tab, clicking 'Execute Live Benchmarks' runs thousands of real-time algorithmic cycles in the browser, demonstrating how AVL Trees deliver up to 580x lower query latency compared to linear array scans."

Part 4: Challenges Faced & Engineering Solutions (Duration: ~1.0 Min)

"During the development of AlgoVision 2.0, I encountered three key engineering challenges:

1. **Challenge 1 (Visualizing AVL Rotations in Real-Time):** Calculating dynamic SVG (x, y) coordinate layouts after multi-level tree rotations was complex. I resolved this by designing a recursive coordinate projection algorithm that scales subtree horizontal offsets (Δx) proportionally to node depth.
2. **Challenge 2 (Circular Money Laundering Detection):** Standard cycle detection can produce false positives on undirected paths. I implemented Tarjan's single-pass Strongly Connected Components algorithm with low-link values and a recursion stack, ensuring zero false alarms.
3. **Challenge 3 (Foreign Exchange Arbitrage Formulation):** Detecting currency arbitrage loops requires multiplicative path maximization. I overcame this by applying a logarithmic transformation $w = -\ln(\text{rate})$, turning the problem into finding negative-weight cycles solvable via the Floyd-Warshall dynamic programming algorithm."

Part 5: Conclusion & Final Reflection (Duration: ~0.5 Min)

"In conclusion, AlgoVision 2.0 bridges theoretical Data Structures & Algorithms with real-world enterprise architectures across Logistics, E-Commerce, and FinTech.

I would like to express my sincere gratitude to Team Launched for this enriching internship opportunity, continuous mentorship, and guidance throughout this journey.

Thank you for watching my capstone project demonstration!"

Recording & Presentation Checklist

- [] **Webcam & Framing:** Head and shoulders visible, good natural lighting facing you, clean background.
- [] **Audio Quality:** Use a decent earphone/headset microphone, speak clearly and at a steady pace.
- [] **Screen Recording Software:** Use OBS Studio, Loom, or Windows Game Bar (Win + G).
- [] **Video Duration:** Keep total video between **5 minutes and 10 minutes**.
- [] **Resolution:** Record at **1080p (1920×1080)** at 30 or 60 FPS.
- [] **Export Format:** MP4 format.
- [] **Google Drive Setting:** Upload the MP4 and verify link sharing is set to **"Anyone with the link can view"**.